

Jinan University
2025 ~ 2026 Academic Year, 2nd. Semester

Java Programming

Course Report

Course code : 60080008
Instructor: Yuxia Sun
Major: Computer Science & Technology
School: International School

Student name : 李荣耀
(p.s. your name on JNU academic system)

Student number : 2024103975

Date of Submission (mm-dd-yyyy): 06/08/2026

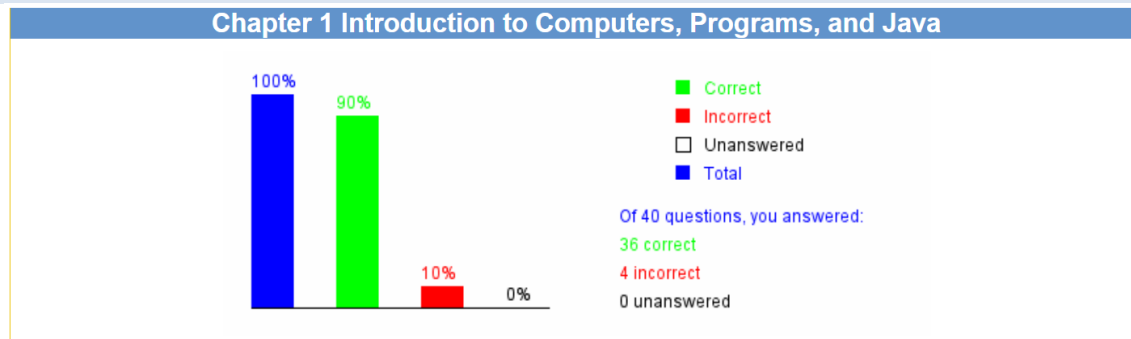
Table of Content

Ch1 Introduction to Computers, Programs, and Java	4
1. Bar-graph of Self-test	4
2. Correction & Learning	4
1) Correction:	4
2) Correction:	4
3) Correction:	4
4) Correction:	5
Ch2 Elementary Programming	5
1. Bar-graph of Self-test	5
2. Correction & Learning	5
1) Correction:	5
2) Correction:	5
3) Correction:	6
4) Correction:	6
5) Correction:	6
Ch3 Selections	7
1. Bar-graph of Self-test	7
2. Correction & Learning	7
1) Correction:	7
2) Correction:	8
Ch4 Mathematical Functions, Characters, and Strings	8
1. Bar-graph of Self-test	8
2. Correction & Learning	9
1) Correction:	9
2) Correction:	9
3) Correction:	9
4) Correction:	9
5) Correction:	10
6) Correction:	10
Ch5 Loops	10
1. Bar-graph of Self-test	10
2. Correction & Learning	11
1) Correction:	11
Ch6 Methods	11
1. Bar-graph of Self-test	11
2. Correction & Learning	11
1) Correction:	11
2) Correction:	12
Ch7 Single-Dimensional Arrays	12
1. Bar-graph of Self-test	12
2. Correction & Learning	12
1) Correction:	12
2) Correction:	13
3) Correction:	13
4) Correction:	14
5) Correction:	14
Ch8 Multidimensional Arrays	14
1. Bar-graph of Self-test	14
2. Correction & Learning	15
Ch9 Objects and Classes	15
1. Bar-graph of Self-test	15
2. Correction & Learning	15
1) Correction:	15
2) Correction:	16

3) Correction:	16
Ch10 Object-Oriented Thinking	16
1. Bar-graph of Self-test	16
2. Correction & Learning	17
1) Correction:	17
2) Correction:	17
Ch11 Inheritance and Polymorphism	17
1. Bar-graph of Self-test	18
2. Correction & Learning	18
1) Correction:	18
2) Correction:	18
3) Correction:	19
4) Correction:	19
Ch12 Exception Handling and Text I/O	20
1. Bar-graph of Self-test	20
2. Correction & Learning	20
1) Correction:	20
2) Correction:	20
Ch13 Abstract Classes and Interfaces	21
1. Bar-graph of Self-test	21
2. Correction & Learning	21
1) Correction:	21
Project I.	22
1. The chosen program' s name and type	23
2. Running results of my modified program	23
3. Program functionalities modified by me	28
4. Test my program with Junit	31
Project II.	34
1. Program Features:	34
2. Java Core Concepts Examples:	37
3. AI Tool Usage:	39
4. Summary on AI-Assisted Programming:	40

Ch1 Introduction to Computers, Programs, and Java

1. Bar-graph of Self-test



2. Correction & Learning

1) Correction:

1.7 NIC is a device to connect a computer to a local area network (LAN).

Learning:

I read the context of “1.2.6 Common Devices”. I know that the dial-up modem, DSL and cable modem are all used to connect Internet through phone line. A network interface card(NIC) is a device to connect a computer to a local area network.

2) Correction:

1.8 Software and programs are instructions to the computer.

Learning:

I search for AI tools. The software is the instructions set to the computer and the programs is the ordered sequence of instructions. The hardware is the carrier of instructions rather than instructions themselves. Keyboards are devices of I/O.

3) Correction:

1.14 Due to security reasons, Java applets cannot run from a Web browser in the new version of Java.

Learning:

I asked for AI tools. Java applets are used in Web browser in early phase. Java applications are common applications by Java. Java servlets are used for server rather than browsers. Micro Edition programs are used for embedded devices.

4) Correction:

1.26 Which JDK command is correct to run a Java application in ByteCode.class? **The Answer: java ByteCode**

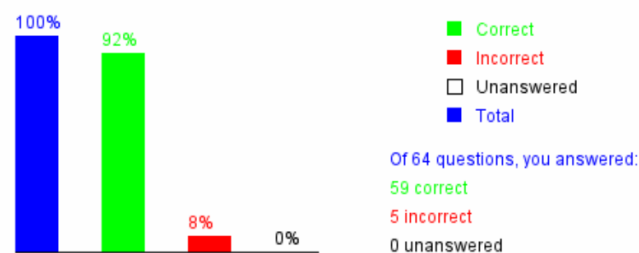
Learning:

I read the book and further understand the running of Java. The instruction `java` pluses the class name rather than the file name.

Ch2 Elementary Programming

1. Bar-graph of Self-test

Chapter 2 Elementary Programming



Please send suggestions and errata to Dr. Liang at y.daniel.liang@gmail.com. Indicate which book and edition you are using. Thanks!

2. Correction & Learning

1) Correction:

2.3 Suppose a Scanner object is created as `Scanner input = new Scanner(System.in)`, the method to read a real number: **input.nextDouble()**

Learning:

I read the answer. The correct method to read a real number is nextDouble() rather than nextdouble().

2) Correction

2.9

2.9 Which of the following are correct ways to declare variables?

- ☒ A. int length; int width;
- ☒ B. int length, width;
- ☐ C. int length; width;
- ☒ D. int length, int width;

Your answer ABD is incorrect



The correct answer is AB

Explanation: Note that a semicolon ends a statement. In B, length and width are both declared as int.

Learning:

I asked for AI tools. I confuse the declaration of variables and the arguments in methods.

3) **Correction:**

2.28 Math.pow(4, 1 / 2) returns **1.0 (wrong answer: 1)**

Learning:

I read the answer. I forget the type of return value of math is double.

4) **Correction:**

2.48 Are the following four statements equivalent?

```
number += 1;
number = number + 1;
number++;
++number;
```

Correct answer: yes

Learning:

I asked for AI tools. The meaning of statements separately are different from the situation they are in expressions.

5) **Correction:**

2.64 Analyze the following code:

```
public class Test {
    public static void main(String[] args) {
        int n = 10000 * 10000 * 10000;
        System.out.println("n is " + n);
    }
}
```

A. The program displays n is 10000000000000.

- B. The result of $10000 * 10000 * 10000$ is too large to be stored in an int variable n. This causes an overflow and the program is aborted.
- C. The result of $10000 * 10000 * 10000$ is too large to be stored in an int variable n. This causes an overflow and the program continues to execute because Java does not report errors on overflow.
- D. The result of $10000 * 10000 * 10000$ is too large to be stored in an int variable n. This causes an underflow and the program is aborted.
- E. The result of $10000 * 10000 * 10000$ is too large to be stored in an int variable n. This causes an underflow and the program continues to execute because Java does not report errors on underflow.

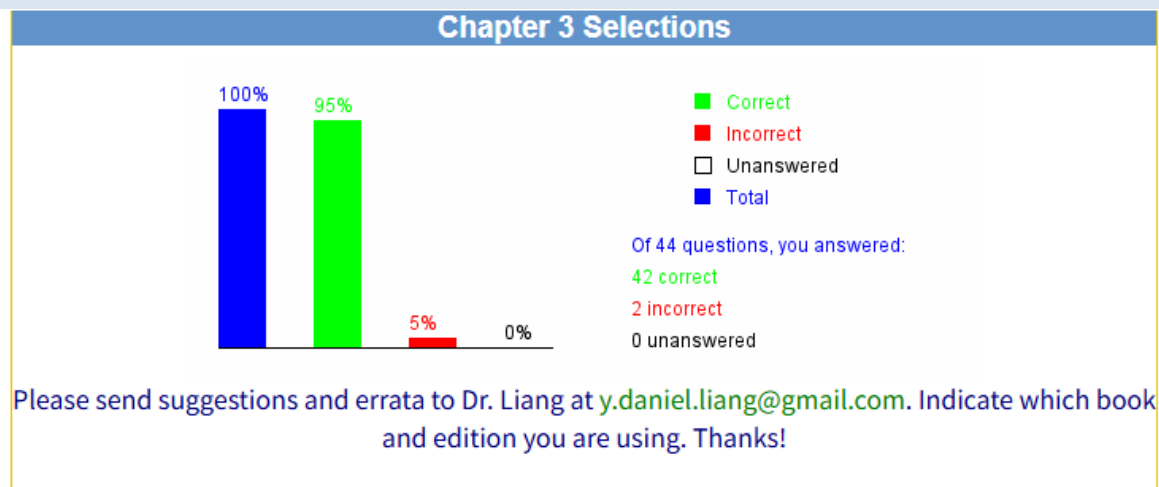
The correct answer: C (Wrong answer: E)

Learning:

I asked for AI tools. Java does not report errors on overflow or underflow for integers. And overflow is for too large(positive). Underflow is for too small(negative).

Ch3 Selections

1. Bar-graph of Self-test



2. Correction & Learning

1) Correction:

3.7 What is the output of the following code?

```
int x = 0;
if (x < 4) {
    x = x + 1;
}
System.out.println("x is " + x);
```

- A. x is 0
- B. x is 1
- C. x is 2
- D. x is 3
- E. x is 4

Correct answer: B (Wrong answer: E)

Learning:

I read the answer. The program is judgement rather than loop.

2) Correction

3.36 Analyze the following program fragment:

```
int x;  
double d = 1.5;  
  
switch (d) {  
    case 1.0: x = 1;  
    case 1.5: x = 2;  
    case 2.0: x = 3;  
}
```

- A. The program has a compile error because the required break statement is missing in the switch statement.
- B. The program has a compile error because the required default case is missing in the switch statement.
- C. The switch control variable cannot be double.
- D. No errors.

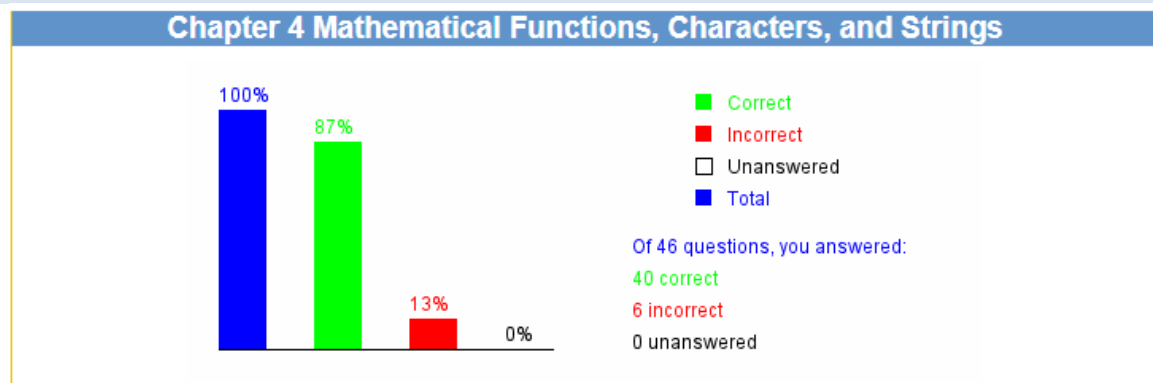
Correct answer: C (Wrong answer D)

Learning:

I read the answer, The switch value cannot be a floating-pointer number.

Ch4 Mathematical Functions, Characters, and Strings

1. Bar-graph of Self-test



2. Correction & Learning

1) Correction:

4.1 To obtain the sine of 35 degrees, use **B**.

- A. `Math.sin(35)`
- B. `Math.sin(Math.toRadians(35))`
- C. `Math.sin(Math.toDegrees(35))`
- D. `Math.sin(Math.toRadian(35))`
- E. `Math.sin(Math.toDegree(35))`

Learning:

I read the answer. I ignore the differences between angles and radians.

2) Correction

4.8 What is `Math rint(3.5)`? **C**

- A. 3.0
- B. 3
- C. 4
- D. 4.0
- E. 5.0

Learning:

I read the answer and search on the Internet. The `rint` returns the nearest integer. When equally close to two integers, returns the nearest even integer.

3) Correction:

4.24 Suppose `Character x = new Character('a')`, **AD** returns true.

- A. `x.equals(new Character('a'))`
- B. `x.compareToIgnoreCase('A')`
- C. `x.equalsIgnoreCase('A')`
- D. `x.equals('a')`
- E. `x.equals("a")`

Learning:

I read the answer and search on the Internet. `equalsIgnoreCase()` and `compareToIgnoreCase()` is the method of `String` rather than `Character`.

4) Correction:

4.26 The expression `"Java " + 1 + 2 + 3` evaluates to **C**.

- A. `Java123`

- B. Java6
- C. Java 123
- D. java 123
- E. Illegal expression

Learning:

I read the answer. When a string ads with a number, the number will be converted into a string.

5) **Correction:**

4.42 The statement `System.out.printf("%3.1f", 1234.56)` outputs **E**.

- A. 123.4
- B. 123.5
- C. 1234.5
- D. 1234.56
- E. 1234.6

Learning:

I read the answer. The rest after the first digit after the decimal point is **rounded up**.

6) **Correction:**

4.43 The statement `System.out.printf("%3.1e", 1234.56)` outputs **D**.

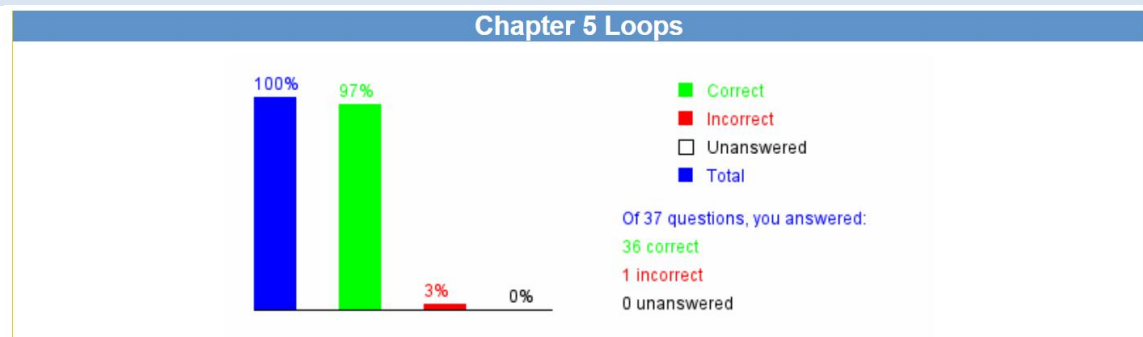
- A. 0.1e+04
- B. 0.123456e+04
- C. 0.123e+04
- D. 1.2e+03
- E. 1.23+03

Learning:

I read the answer. It is the similar mistake as the last mistake. The rest after corresponding digits after decimal point should be rounded up.

Ch5 Loops

1. Bar-graph of Self-test



2. Correction & Learning

1) Correction:

5.23 To add $0.01 + 0.02 + \dots + 1.00$, what order should you use to add the numbers to get better accuracy? **A**

A.add 0.01, 0.02, ..., 1.00 in this order to a sum variable whose initial value is 0.

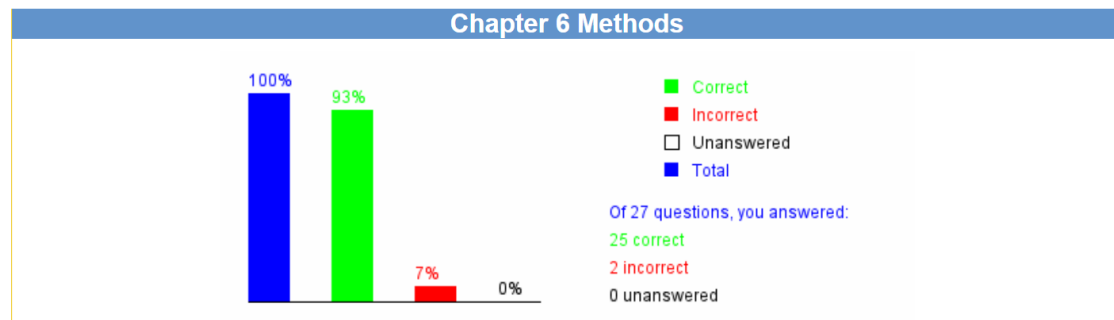
B.add 1.00, 0.99, 0.98, ..., 0.02, 0.01 in this order to a sum variable whose initial value is 0.

Learning:

I read the answer and asked for AI tools. The addition of two floating numbers will be more accurate when they are closer. Since in Java they are IEEE 754 floating numbers, which will transform number into scientific notation and then align the exponents. If they are large difference between their exponents, the smaller number will lose the last digits.

Ch6 Methods

1. Bar-graph of Self-test



2. Correction & Learning

1) Correction:

6.13 Given the following method, what is the output of the call `nPrint('a', 4)`? **D (Wrong answer: B)**

```
static void nPrint(String message, int n) {
    while (n > 0) {
        System.out.print(message);
        n--;
    }
}
```

A. aaaaa

- B. aaaa
- C. aaa
- D. invalid call

Learning:

I read the answer. I ignore the 'a' is the type of char rather than String. Thus the calling is invalid.

2) Correction:

6.27 A (Wrong answer: D) is a simple but incomplete version of a method.

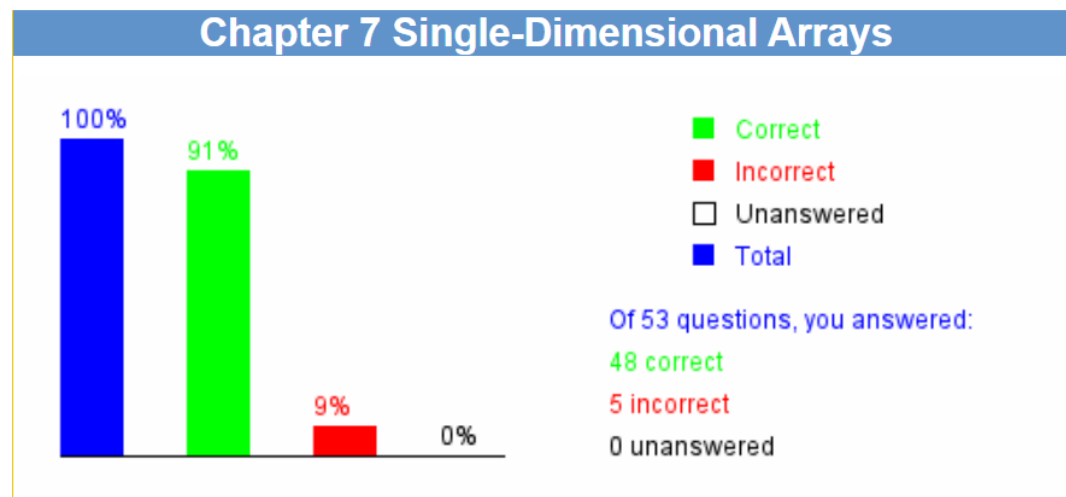
- A. A stub
- B. A main method
- C. A non-main method
- D. A method developed using top-down approach

Learning:

I read the book and searched on the Internet. A stub normally only includes method signature and simple return statement. It is a simple but incomplete version of a method. A method developed using top-down approach is only describing the process of developing.

Ch7 Single-Dimensional Arrays

1. Bar-graph of Self-test



2. Correction & Learning

1) Correction:

7.17 What is output of the following code: **B (Wrong answer: D)**

```
public class Test {
```

```
public static void main(String[] args) {
    int[] x = {120, 200, 016};
    for (int i = 0; i < x.length; i++)
        System.out.print(x[i] + " ");
}
```

A. 120 200 16

B. 120 200 14

C. 120 200 20

D. 016 is a compile error. It should be written as 16.

Learning:

I read the answer and searched on the Internet. The prefix 0 indicates the number is in octal in both Java and C.

2) Correction

7.29 Show the output of the following code: **D (Wrong answer: C)**

```
public class Test {
    public static void main(String[] args) {
        int[] x = {1, 2, 3, 4, 5};
        increase(x);

        int[] y = {1, 2, 3, 4, 5};
        increase(y[0]);

        System.out.println(x[0] + " " + y[0]);
    }

    public static void increase(int[] x) {
        for (int i = 0; i < x.length; i++)
            x[i]++;
    }

    public static void increase(int y) {
        y++;
    }
}
```

A. 0 0

B. 1 1

C. 2 2

D. 2 1

E. 1 2

Learning:

I read the answer. I ignored that it should pass reference to modify indexed variable. The value of y[0] outside the method is not changed.

3) Correction:

7.38 Which of the following declarations are correct? **DE (Wrong answer: ABDE)**

A. public static void print(String... strings, double... numbers)

B. public static void print(double... numbers, String name)

- C. public static double... print(double d1, double d2)
- D. public static void print(double... numbers)
- E. public static void print(int n, double... numbers)

Learning:

I read the answer and search on the Internet. The rules of various arguments are: there exists at most one variable-length parameter; It must be the last one argument; the method cannot return a variable-length parameter.

4) Correction:

7.50 Which code fragment would correctly identify the number of arguments passed via the command line to a Java application, excluding the name of the class that is being invoked? **A (Wrong answer: C)**

- A. int count = args.length;
- B. int count = args.length - 1;
- C. int count = 0; while (args[count] != null) count ++;
- D. int count=0; while (!(args[count].equals(""))) count ++;

Learning:

I read the answer and asked for AI tools. I made a mistake when reading the question. It excludes the name of the class.

5) Correction:

7.51 Which correctly creates an array of five empty Strings? **B (Wrong answer: D)**

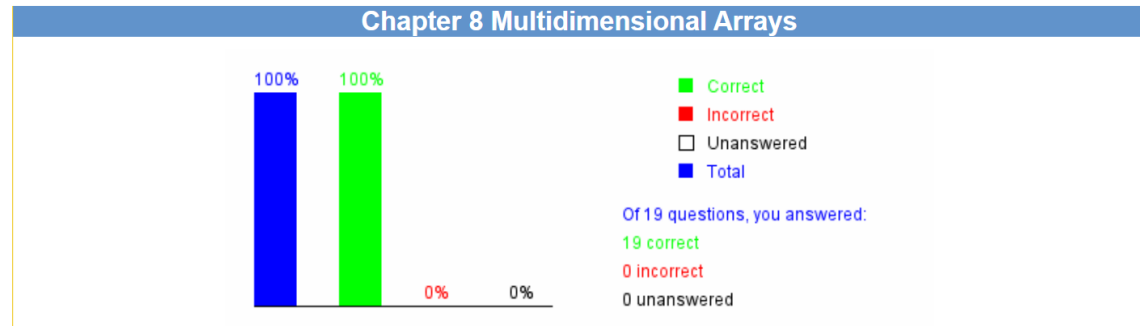
- A. String[] a = new String [5];
- B. String[] a = {"", "", "", "", ""};
- C. String[5] a;
- D. String[] a = new String [5]; for (int i = 0; i < 5; a[i++] = null);

Learning:

I read the answer and asked for AI. The empty strings are different from null. The type of String is a type of reference type. It points to null if no object reference. It points to "" if there exists one string object whereas it is empty.

Ch8 Multidimensional Arrays

1. Bar-graph of Self-test



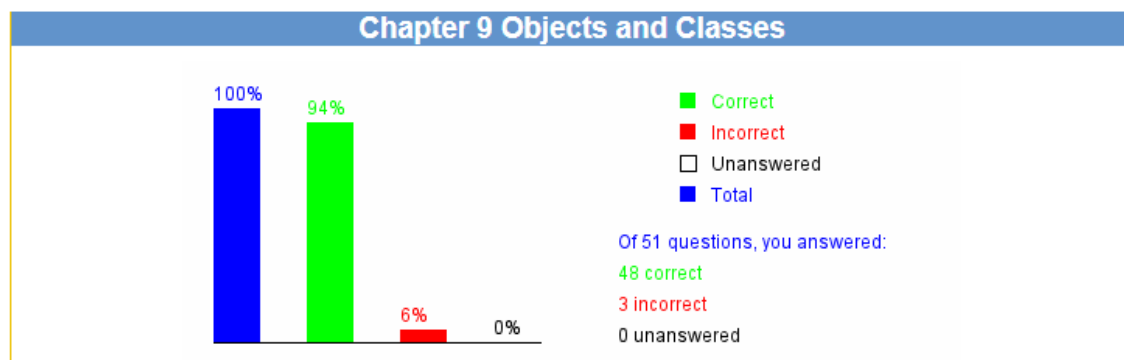
2. Correction & Learning

Learning:

I deepen the understanding of multidimensional arrays. It is essentially higher pointer to lower pointer. When initializing, the number of last level pointer is not necessarily clear.

Ch9 Objects and Classes

1. Bar-graph of Self-test



2. Correction & Learning

1) Correction:

9.24 To declare a constant MAX_LENGTH as a member of the class, you write

- A. final static MAX_LENGTH = 99.98;
- B. final static float MAX_LENGTH = 99.98;
- C. static double MAX_LENGTH = 99.98;
- D. final double MAX_LENGTH = 99.98;
- E. final static double MAX_LENGTH = 99.98;

Learning:

I read the answer and asked for AI tools. The constant in the class should be static to improve the efficiency and make it independent to object.

2) Correction:

9.25 Analyze the following code.

```
public class Test {  
    public static void main(String[] args) {  
        int n = 2;  
        xMethod(n);  
  
        System.out.println("n is " + n);  
    }  
  
    void xMethod(int n) {  
        n++;  
    }  
}
```

- A. The code has a compile error because xMethod does not return a value.
- B. The code has a compile error because xMethod is not declared static.
- C. The code prints n is 1.
- D. The code prints n is 2.
- E. The code prints n is 3.

Learning:

I search for AI tools. The static method cannot invoke the non-static method directly.

3) Correction:

9.28 What code may be filled in the blank without causing syntax or runtime errors:

```
public class Test {  
    java.util.Date date;  
  
    public static void main(String[] args) {  
        Test test = new Test();  
        System.out.println(_____);  
    }  
}
```

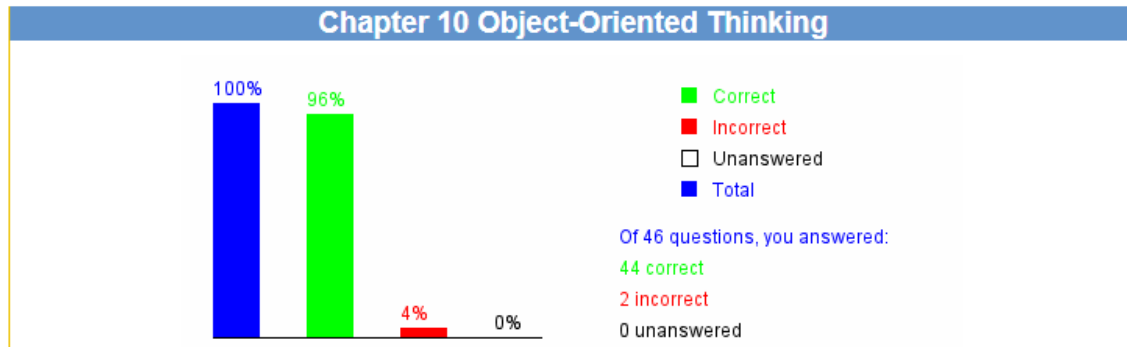
- A. test.date
- B. date
- C. test.date.toString()
- D. date.toString()

Learning:

I checked the answer and asked for AI tools. Date is an instance variable and cannot be accessed from static context. System.out.println(test.date) will invoke println(Object) method and output null. If invoking toString(), it will throw NullPointerException.

Ch10 Object-Oriented Thinking

1. Bar-graph of Self-test



2. Correction & Learning

1) Correction:

10.6 Which of the following statements are correct?

- A. Integer.parseInt("12", 2);
- B. Integer.parseInt(100);
- C. Integer.parseInt("100");
- D. Integer.parseInt(100, 16);
- E. Integer.parseInt("345", 8);

Learning:

I read the answer and asked for AI tools. The first parameter is a string, and the second parameter is the scale (2, 8, 10, 16, etc.) For A, "12" is not a binary string.

2) Correction:

10.20 What is the output of the following code?

```
public class Test {
    public static void main(String[] args) {
        String s1 = "Welcome to Java!";
        String s2 = "Welcome to Java!";

        if (s1 == s2)
            System.out.println("s1 and s2 reference to the same String
object");
        else
            System.out.println("s1 and s2 reference to different String
objects");
    }
}
```

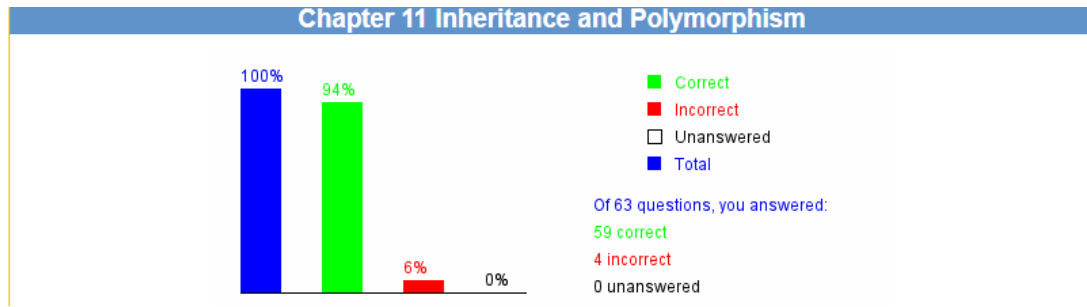
- A. s1 and s2 reference to the same String object
- B. s1 and s2 reference to different String objects

Learning:

I read the answer and searched for AI tools. Since strings are immutable and are ubiquitous in programming, to improve efficiency and save memory, the JVM uses a unique instance for string literals with the same character sequence. Such an instance is called interned.

Ch11 Inheritance and Polymorphism

1. Bar-graph of Self-test



2. Correction & Learning

1) Correction:

11.20 Which of the following are Java keywords?

- A. instanceof
- B. instanceof
- C. cast
- D. casting

Learning:

I read the answer. I ignore the keywords are all in lowercase.

2) Correction:

11.33 Analyze the following code.

```
// Program 1:
public class Test {
    public static void main(String[] args) {
        Object a1 = new A();
        Object a2 = new A();
        System.out.println(a1.equals(a2));
    }
}

class A {
    int x;

    public boolean equals(Object a) {
        return this.x == ((A)a).x;
    }
}

// Program 2:
public class Test {
    public static void main(String[] args) {
        Object a1 = new A();
        Object a2 = new A();
        System.out.println(a1.equals(a2));
    }
}
```

```

    }

    class A {
        int x;

        public boolean equals(A a) {
            return this.x == a.x;
        }
    }

```

- A. Program 1 displays true and Program 2 displays true
- B. Program 1 displays false and Program 2 displays true
- C. Program 1 displays true and Program 2 displays false
- D. Program 1 displays false and Program 2 displays false

Learning:

I read the answer and searched for AI tools. Since the signature of equals() in the class A is not the same as the equals() in the Object class, it is not overridden. It will still invoke the equals() in the Object class.

3) **Correction:**

11.44 What is the output of the following code?

```

ArrayList<java.util.Date> list = new ArrayList<java.util.Date>();
java.util.Date d = new java.util.Date();
list.add(d);
list.add(d);
System.out.println((list.get(0) == list.get(1)) + " "
    + (list.get(0)).equals(list.get(1)));

```

- A. true false
- B. false true
- C. true true
- D. false false

Learning:

I asked for AI tools. The timestamp in the java.util.Date is not dynamic. That is, when creating the instance, the timestamp will be set as a fixed value. So the values for two addition are the same.

4) **Correction:**

11.52 Analyze the following code:

```

Double[] array = {1, 2, 3};
ArrayList<Double> list = new ArrayList<>(Arrays.asList(array));
System.out.println(list);

```

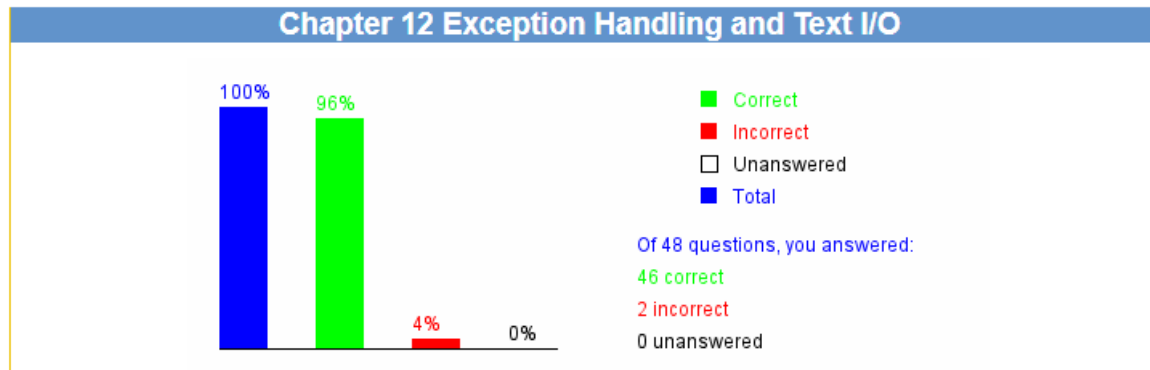
- A. The code is correct and displays [1, 2, 3].
- B. The code is correct and displays [1.0, 2.0, 3.0].
- C. The code has a compile error because an integer such as 1 is automatically converted into an Integer object, but the array element type is Double.
- D. The code has a compile error because asList(array) requires that the array elements are objects.

Learning:

I read the book and run the code actually. The int type cannot be automatically boxed into Double type.

Ch12 Exception Handling and Text I/O

1. Bar-graph of Self-test



2. Correction & Learning

1) Correction:

12.15 Analyze the following code:

```
public class Test {
    public static void main(String[] args) throws MyException {
        System.out.println("Welcome to Java");
    }
}

class MyException extends Error {
}
```

- A. You should not declare a class that extends Error, because Error raises a fatal error that terminates the program.
- B. You cannot declare an exception in the main method.
- C. You declared an exception in the main method, but you did not throw it.
- D. The program has a compile error.

Learning:

I read the answer. The Error is handled by the system. Generally it is not inherited by users.

2) Correction:

12.24 What is displayed on the console when running the following program?

```
public class Test {
    public static void main(String[] args) {
        try {
            System.out.println("Welcome to Java");
            int i = 0;
            double y = 2.0 / i;
        }
    }
}
```

```

        System.out.println("Welcome to HTML");
    }
    finally {
        System.out.println("The finally clause is executed");
    }
}
}

```

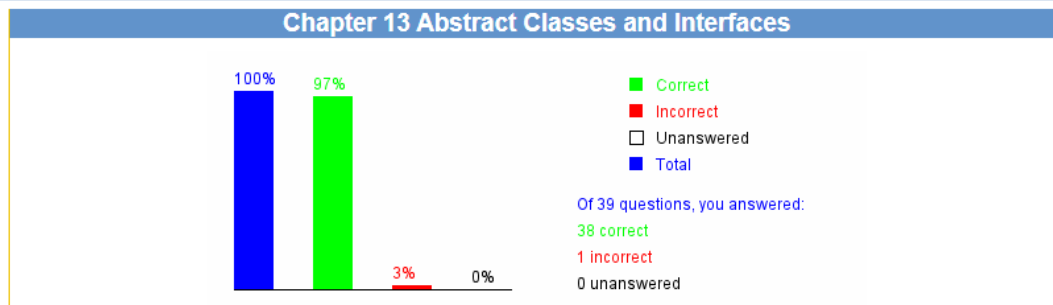
- A. Welcome to Java.
- B. Welcome to Java followed by The finally clause is executed in the next line.
- C. The program displays three lines: Welcome to Java, Welcome to HTML, The finally clause is executed.
- D. None of the above.

Learning:

I read the answer and searched for AI tools. The floating number divided by 0 will return INFINITY rather than raise an exception.

Ch13 Abstract Classes and Interfaces

1. Bar-graph of Self-test



2. Correction & Learning

1) Correction:

13.6 What is the output of running class Test?

```

public class Test {
    public static void main(String[] args) {
        new Circle9();
    }
}

public abstract class GeometricObject {
    protected GeometricObject() {
        System.out.print("A");
    }

    protected GeometricObject(String color, boolean filled) {
        System.out.print("B");
    }
}

```

```
public class Circle9 extends GeometricObject {
    /** No-arg constructor */
    public Circle9() {
        this(1.0);
        System.out.print("C");
    }

    /** Construct circle with a specified radius */
    public Circle9(double radius) {
        this(radius, "white", false);
        System.out.print("D");
    }

    /** Construct a circle with specified radius, filled, and color */
    public Circle9(double radius, String color, boolean filled) {
        super(color, filled);
        System.out.print("E");
    }
}
```

- A. ABCD
- B. BACD
- C. CBAE
- D. AEDC
- E. BEDC

Learning:

I asked for AI tools. Only when the constructor of subclass does not invoke the constructor of superclass compiler will invoke the super() in default. If the subclass invokes the super explicitly, it won't invoke super().

Project I.

<https://www.interviewbit.com/blog/java-projects/>

- Java Projects For Beginners
 - 1. Password Generator using Java
 - 2. Online Survey System
 - 3. Online Resume Builder
 - 4. Snake Game using Java
- Intermediate Java Projects With Source Code
 - 5. Data Visualization Software
 - 6. Electricity Billing System
 - 7. Web Medical Management System
 - 8. Supply Chain Management System
 - 9. Exam Seating Arrangement System in Java
 - 10. Wordcount Tools in Java
- Core Java Projects With Source Code
 - 11. Create a Consumer Relationship Management System
 - 12. bFit Cognitive and Memory Testing Game
 - 13. Network Packet Sniffer Analyzer Software
 - 14. Internet Service Provider Automation System
 - 15. Create a Criminal Face Detection System

Choose one of the above 15 programs.

Modify the program's GUI and some functionalities, and describe which parts are your modification. Show the running results of your modified program.
And test your program using Junit.

1. The chosen program's name and type

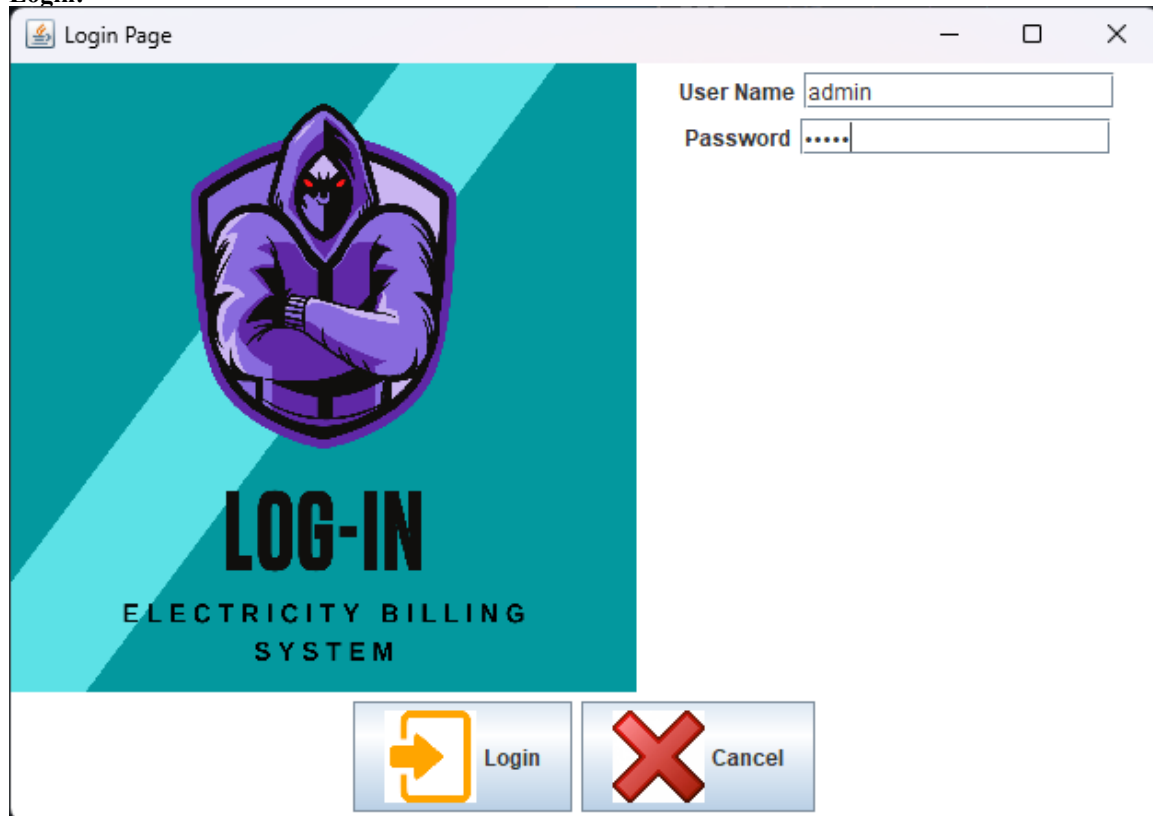
6. Electricity Billing System (Intermediate Java Projects)

The original system allows an administrator to manage customer information, record meter readings, calculate bills based on a fixed rate per unit, and generate invoices. It uses a MySQL database for data persistence and a Swing-based graphical user interface.

2. Running results of my modified program

(1) Screenshot of the running result

Login:



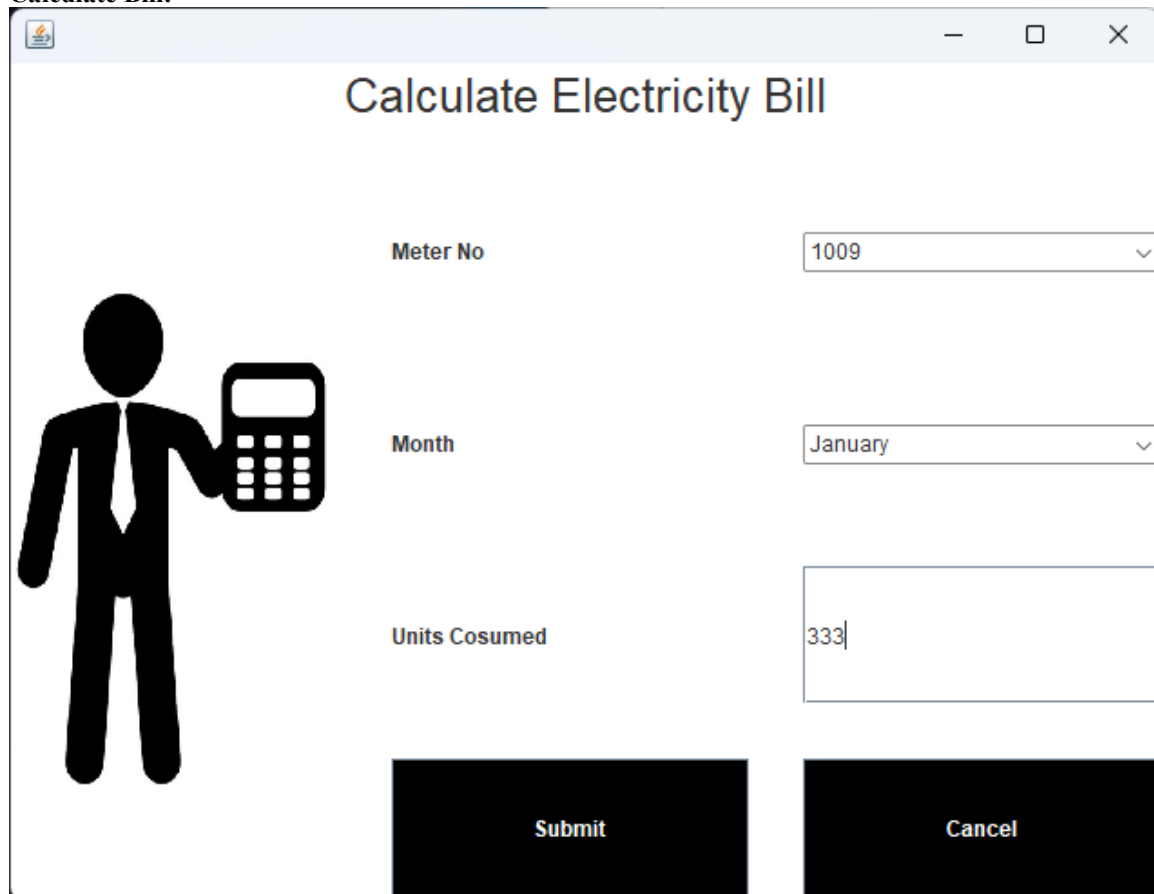
Main page:



Add Customer:

The screenshot shows the 'Add Customer' form window. The title bar reads 'Add Customer'. On the left side of the form is a black silhouette of a person holding a white document. To the right of the silhouette are seven input fields, each with a label to its left: 'Name' (containing 'John Lennon'), 'Meter No' (containing '1009'), 'Address' (containing 'Here, there and everywhere'), 'State' (containing 'GOOD'), 'City' (containing 'Imagine'), 'Email' (containing 'inmylife@foxmail.com'), and 'Phone Number' (containing '1145141919810D'). At the bottom of the form are two large black buttons: 'Submit' on the left and 'Cancel' on the right.

Calculate Bill:



Calculate Electricity Bill

Meter No: 1009

Month: January

Units Consumed: 333

Submit **Cancel**

消息

0-100: 300
101-300: $200 \times 5.0 = 1000$
301-333: $33 \times 7.0 = 231.0$
total amount: 1765

确定

消息

Bill Updated

确定

Details:

Jinan University– Java Programming Course Report

[illegible]

Generate Bill:

Generate Bill 1009 January

Reliance Power Limited
ELECTRICITY BILL FOR THE MONTH OF Jan.

Customer Name: John Lennon
Meter Number: 1009
Address: Here, there and everywhere
State: GOOD
City: Imagine
Email: inmylife@foxmail.com
Phone Number 1145141919810D

Current Month :	333
Units Consumed:	January
Total Charges :	1765

TOTAL PAYABLE : 1765

Email Bill

Generate Bill



(2) The modified parts of the GUI

pay_bill:

Modified the logic of closing the window to avoid the crash of the whole system. Enhance the security of the system.

calculate_bill:

Enhanced the window with a detailed text area that shows the calculation steps and an updated text after submission.

generate_bill:

Added an email bill button in the window, allowing the user to send the bill PDF to the customer's registered mail address.

3. Program functionalities modified by me

(1) Description of the modified functionalities

- **conn.java**
To locate the exception or error more accurately when connecting to the database, we add some statement to ensure the process and modify the exception report to '*e.printStackTrace()*'. Moreover, we modified the driver to '*com.mysql.cj.jdbc.Driver*' to make it compatible for more recent version of MySQL.
- **pay_bill.java**
JEditorPane is an outdated Swing element and it can only render simple HTML and webpage. So it always shows "Could not load". Furthermore, the class uses '*EXIT_ON_CLOSE*'. It will cause the JVM exit if we see the error message of the window. Here, we simply modify the function to '*DISPOSE_ON_CLOSE*' to only close current window and omit further optimize of the embedding of webpage.
- **calculate_bill.java**
Optimize the rules of price. We use dynamic pricing based on monthly consumption: 0-100 units is 3 Rs/unit, 101-300 units is 5 Rs/unit and 301+ is 7 Rs/unit. Calculation is shown to the user. Additionally, original code has not distinguished the buttons of submit and cancel. We modify it to use cancel to close the window.
- **generate_bill.java**
After generating a bill, the user can click "Email Bill" to send the bill to the customer's email address using '*javax.mail*'.

(2) The total number of the SourceCode Lines created by me

Total new/modified Java code lines is about 100 lines, including changes to pay bill, calculate bill, generate bill and connection.

(3) List parts of the SourceCode Lines (less than one page) created by me

conn.java:

```
try {  
  
    System.out.println("Loading Driver...");  
  
    Class.forName("com.mysql.cj.jdbc.Driver");
```

```
System.out.println("Connecting DB...");

c = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/ebs",
    "root",
    "2.8682722985");

System.out.println("Creating Statement...");

s = c.createStatement();

System.out.println("SUCCESS");

} catch (Exception e) {
    e.printStackTrace();
}
```

pay_bill.java:

```
JScrollPane scrollPane = new JScrollPane(j);
setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
getContentPane().add(scrollPane);
```

generate_bill.java:

```
try {
    // handle the action performed by the button of email bill
    if (ae.getSource() == b2) {
        String recipient = getCustomerEmail();
        if (recipient == null || recipient.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Customer email
not found for this meter number!");
            return;
        }
        String subject = "Electricity Bill for " +
c2.getSelectedItem();
        String body = t1.getText();
        sendEmail(recipient, subject, body);
    }
}
```

...

```
// method to send email
private void sendEmail(String to, String subject, String body) {
    try {
        Properties props = new Properties();
        props.put("mail.smtp.host", "smtp.qq.com");
        props.put("mail.smtp.port", "465");
        props.put("mail.smtp.auth", "true");
```

```

        props.put("mail.smtp.ssl.enable", "true");
        Session session = Session.getInstance(props, new
javax.mail.Authenticator() {
            protected javax.mail.PasswordAuthentication
getPasswordAuthentication() {
                return new
javax.mail.PasswordAuthentication("lirongyao6310@qq.com",
"myijqrjivvndfcb");
            }
        });
        MimeMessage msg = new MimeMessage(session);
        msg.setFrom(new InternetAddress("lirongyao6310@qq.com"));
        msg.setRecipient(Message.RecipientType.TO, new
InternetAddress(to));
        msg.setSubject(subject);
        msg.setText(body);
        Transport.send(msg);
        JOptionPane.showMessageDialog(this, "Bill sent to " + to);
    } catch (Exception e) {
        e.printStackTrace();
        JOptionPane.showMessageDialog(this, "Failed to send email");
    }

}

private String getCustomerEmail() {
    try {
        conn c = new conn();
        ResultSet rs = c.s.executeQuery("select email from emp where
meter_number= '" + c1.getSelectedItemAt() + "'");
        if (rs.next())
            return rs.getString("email");
    } catch (Exception e) {
        e.printStackTrace();
    }
    return "";
}
}

```

calculate_bill.java:

```

// cancel to close the frame
if (ae.getSource() == b2) {
    dispose();
    return;
}

```

```

// modify calculating bill logic

```

```
int units = p1;
String slabDetail = ""; // Initialize slabDetail variable

double total = calculateTieredAmount(units, slabDetail);
int p3 = (int) total;

// Method to calculate tiered amount with fixed fees
public static double calculateTieredAmount(int units, StringBuilder
slabDetail) {
    double amount = 0;
    slabDetail.setLength(0);
    if (units <= 100) {
        amount = units * 3.0;
        slabDetail.append("0-100: ").append(units).append(" x 3.0 =
").append(amount).append("\n");
    } else if (units <= 300) {
        amount = 100 * 3.0 + (units - 100) * 5.0;
        slabDetail.append("0-100: 300\n101-").append(units).append(":
").append(units - 100).append(" x 5.0 = ").append((units - 100) *
5.0).append("\n");
    } else {
        amount = 100 * 3.0 + 200 * 5.0 + (units - 300) * 7.0;
        slabDetail.append("0-100: 300\n101-300: 1000\n").append("301-
").append(units).append(": ")
            .append(units - 300).append(" x 7.0 = ").append((units
- 300) * 7.0).append("\n");
    }

    double fixedFees = 50 + 12 + 102 + 20 + 50;
    slabDetail.append("fixed fee: ").append(fixedFees);
    // modify to include fixed fees
    return amount + fixedFees;
}
```

4. Test my program with Junit.

(1) Screenshots (less than 5 images) of testing classes in my program with Junit

```
Thanks for using JUnit! Support its development at https://junit.org/sponsoring

Loading Driver...
Connecting DB...
Creating Statement...
SUCCESS
Loading Driver...
Connecting DB...
Creating Statement...
SUCCESS
.
+-- JUnit Jupiter [OK]
| +-- GenerateBillTest [OK]
| | '-- testGetCustomerEmail() [OK]
| '-- CalculateBillTest [OK]
| +-- testSecondSlab() [OK]
| +-- testThirdSlab() [OK]
| '-- testFirstSlab() [OK]
+-- JUnit Vintage [OK]
'-- JUnit Platform Suite [OK]

Test run finished after 640 ms
[      5 containers found      ]
[      0 containers skipped    ]
[      5 containers started    ]
[      0 containers aborted    ]
[      5 containers successful ]
[      0 containers failed     ]
[      4 tests found          ]
[      0 tests skipped         ]
[      4 tests started         ]
[      0 tests aborted         ]
[      4 tests successful      ]
[      0 tests failed          ]
```

(2) list parts of source lines (less than one page) in your test class

GenerateBillTest.java:

```
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

public class GenerateBillTest {
    private generate_bill window;
    private conn testConn;
    private final String TEST_METER = "1001";

    @BeforeEach
    void setUp() throws Exception {
```



```
        testConn = new conn();
        testConn.s.executeUpdate(
            "INSERT INTO emp (meter_number, email) VALUES ('" +
TEST_METER + "', 'test@example.com') " +
            "ON DUPLICATE KEY UPDATE
email='test@example.com'");
        window = new generate_bill();
        window.c1.select(TEST_METER);
    }

    @AfterEach
    void tearDown() throws Exception {
        testConn.s.executeUpdate("UPDATE emp SET email = NULL WHERE
meter_number='" + TEST_METER + "'");
        testConn.c.close();
    }

    @Test
    void testGetCustomerEmail() {
        String email = window.getCustomerEmail();
        assertEquals("test@example.com", email);
    }
}
```

CalculateBillTest.java:

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class CalculateBillTest {

    @Test
    void testFirstSlab() {
        StringBuilder slabDetail = new StringBuilder();
        double amount = calculate_bill.
        calculateTieredAmount(50, slabDetail);
        assertEquals(384.0, amount, 0.01);
    }

    @Test
    void testSecondSlab() {
        StringBuilder slabDetail = new StringBuilder();
        double amount = calculate_bill.calculateTieredAmount(200,
slabDetail);
        assertEquals(1034.0, amount, 0.01);
    }
}
```

```
@Test
void testThirdSlab() {
    StringBuilder slabDetail = new StringBuilder();
    double amount = calculate_bill.calculateTieredAmount(400,
slabDetail);
    assertEquals(2234.0, amount, 0.01);
}
}
```

Project II.

Design and implement a functionally comprehensive Java program with practical application value using AI-assisted technology. Explore the use of AI tools (such as GitHub Copilot, Cursor, ChatGPT, TabNine, Codeium, etc.) for design, code generation, and testing.

1. Program Features:

(1) Textual description of the features

This system is a Student Grade Management System designed for teachers, using Java Swing + JDBC + MySQL. It implements the following core functions:

- Student Information Management: Supports adding, modifying, deleting, and querying student information (student ID, name, class). Data is persistently stored in a MySQL database.
- Grade Entry & Editing: Displays all students and all courses in a two-dimensional table. Teachers can directly enter or modify grades in the table, and the system automatically saves changes to the database.
- Grade Statistics & Charts: Dynamically generates bar charts of grade distribution for each course based on the actual grade data in the database. Counts are grouped by score ranges (0-59, 60-69, 70-79, 80-89, 90-100), using the JFreeChart library.
- Export Grade Report: Supports exporting the current student grade table as a PDF file (using iText library), convenient for printing or archiving.
- Strategy Pattern for Grade Calculation: Demonstrates two grade calculation strategies – weighted average and best score – which can be flexibly switched in the code.

(2) Screenshots of the main GUI interface

Main Window:

学生成绩管理系统		
学生管理 成绩管理 统计分析 导出		
学号	姓名	班级
2021001	Amy	CS-A
2021002	Bob	CS-A
2021003	Charlie	CS-B
2021004	David	CS-B

Add Student Dialog:

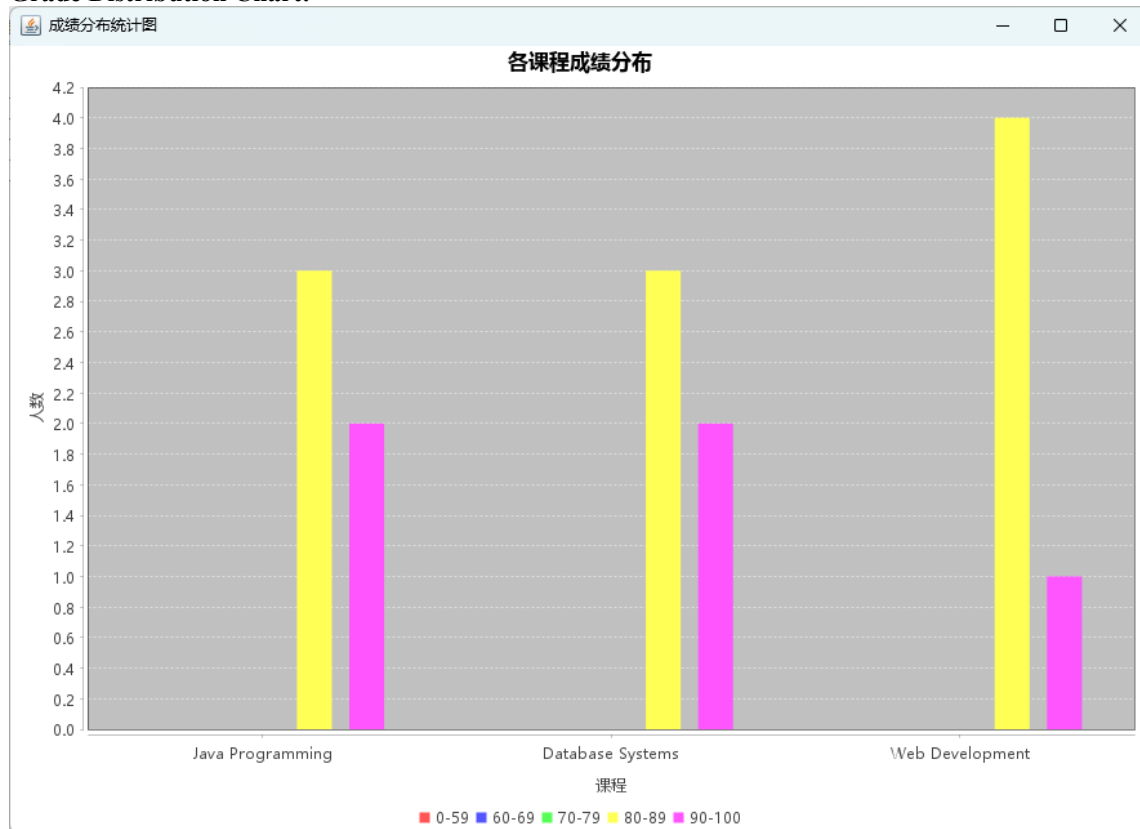
添加学生	
学号:	2021005
姓名:	Lennon
班级:	CS-B
保存	取消

Grade Entry Window:

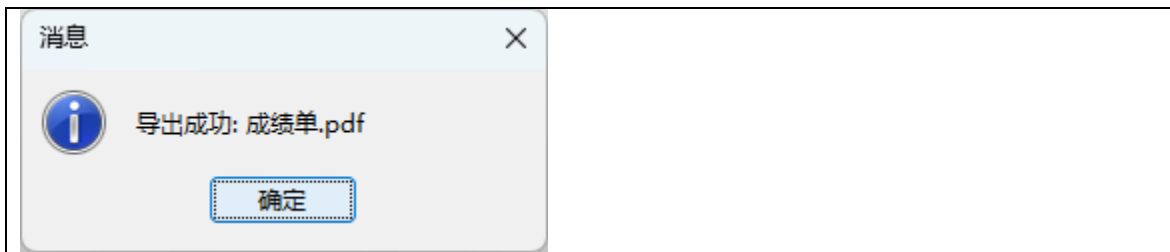
成绩录入			
学生姓名	Java Programming	Database Systems	Web Development
Amy	80.0	83.0	81.0
Bob	81.0	83.0	83.0
Charlie	81.0	89.0	89.0
David	93.0	91.0	83
Lennon	91	91	91

保存所有修改

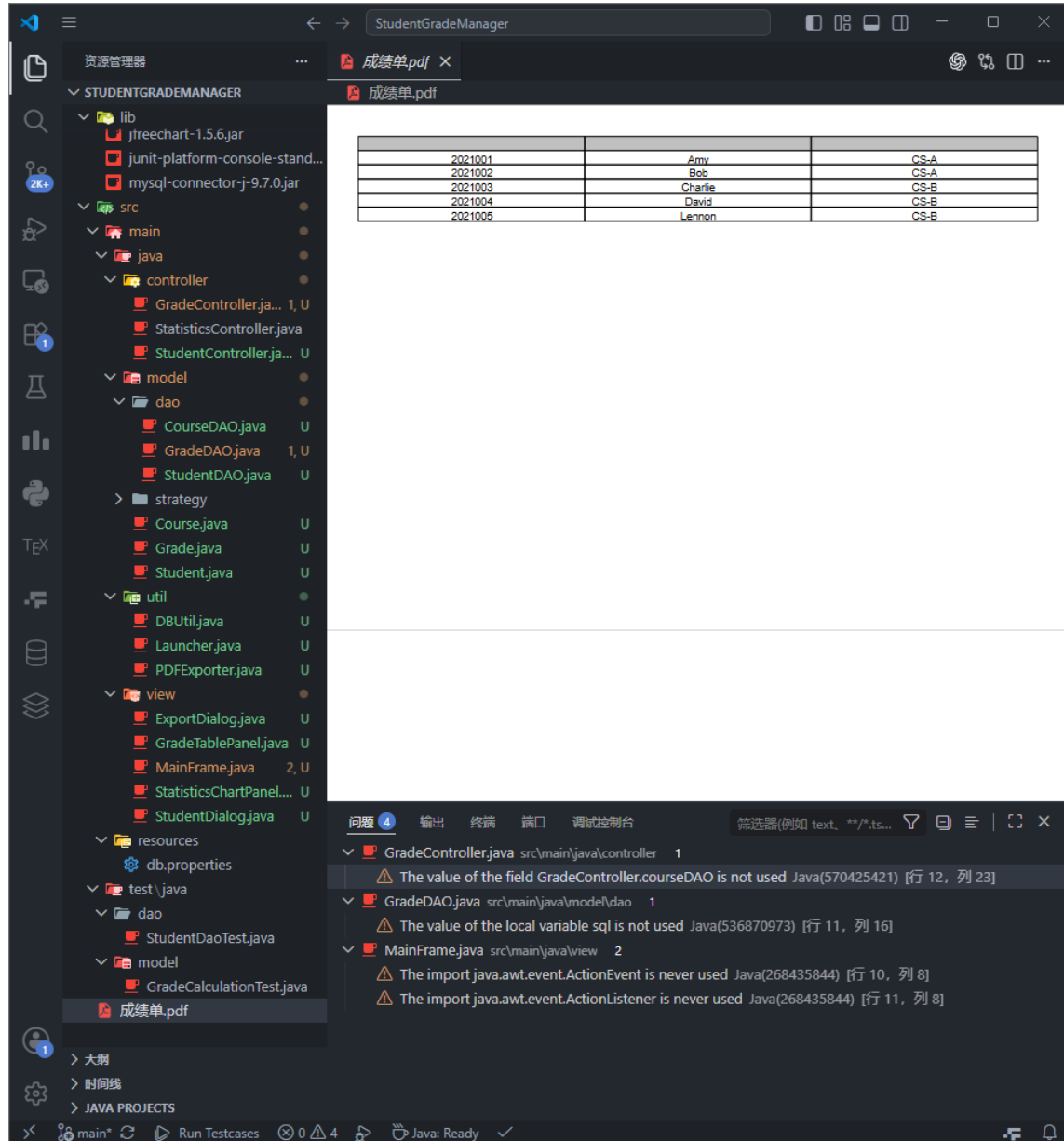
Grade Distribution Chart:



PDF Export Success Prompt:



PDF:



2. Java Core Concepts Examples:

Provide code examples demonstrating the following concepts:

(1) Inheritance

Example: The custom dialog `StudentDialog` extends `JDialog` and reuses the modal window behavior of the parent class.

```

public class StudentDialog extends JDialog {
    public StudentDialog(JFrame parent, Student student) {
        super(parent, student == null ? "Add Student" : "Edit Student", true);
        // ... component initialization
    }
}

```

Explanation: By calling `super(parent, title, modal)`, the constructor sets the parent frame, title, and modal property, avoiding repetitive low-level window logic.

(2) Polymorphism

Example: The `GradeCalculationStrategy` interface and its two implementations `WeightedAverageStrategy` and `BestScoreStrategy`. In `GradeController`, the strategy can be switched at runtime.

```

public interface GradeCalculationStrategy {
    double calculate(double[] scores, int[] weights);
}

public class WeightedAverageStrategy implements GradeCalculationStrategy {
    @Override
    public double calculate(double[] scores, int[] weights) {
        // weighted average calculation
    }
}

// In the controller
private GradeCalculationStrategy strategy;
public void setStrategy(GradeCalculationStrategy strategy) {
    this.strategy = strategy;
}

```

Explanation: The `setStrategy` method accepts the interface type, enabling any implementation to be passed at runtime – a typical example of polymorphism (one interface, multiple implementations).

(3) Exception Handling

Example: Database operations catch `SQLException`, either re-throw it as a runtime exception or display a user-friendly error message in the GUI.

```

public List<Student> findAll() {
    try {

```

```

        // database query
    } catch (SQLException e) {
        System.err.println("Database query failed: " + e.getMessage());
        throw new DataAccessException("Error fetching student data", e);
    }
}

// In the GUI layer
try {
    List<Student> students = studentController.getAllStudents();
} catch (DataAccessException e) {
    JOptionPane.showMessageDialog(this, "Failed to load data: " + e.getMessage());
}

```

Explanation: The low-level `SQLException` is wrapped into a custom unchecked `DataAccessException`, simplifying the call chain while providing user feedback and preventing program crashes.

(4) GUI

Example: The main window `MainFrame` uses `JMenuBar`, `JMenu`, and `JMenuItem` to build the menu bar, and `JTable` with `DefaultTableModel` to display student data. Actions are handled via `ActionListener`.

```

// Creating menus
JMenu studentMenu = new JMenu("Student Management");
JMenuItem addItem = new JMenuItem("Add Student");
addItem.addActionListener(e -> showStudentDialog(null));
studentMenu.add(addItem);

// Table model
tableModel = new DefaultTableModel(new String[]{"Student ID", "Name", "Class"}, 0);
studentTable = new JTable(tableModel);
add(new JScrollPane(studentTable), BorderLayout.CENTER);

```

Explanation: Standard Swing components are used with `BorderLayout` to achieve a menu-driven main interface and a data table.

3. AI Tool Usage:

(describe at least one AI tool)

(1) Name of the AI tool

ChatGPT-4o

(2) Functions and your usage scenarios (such as feature design, code generation, or test case generation)

- **Code Generation:** Produced complete Java classes from natural language descriptions, such as `GradeController`, `StatisticsController`, and `PDFExporter`.
- **Debugging Assistance:** Provided error-fixing suggestions (e.g., `NullPointerException`, classpath configuration, font issues) when presented with error stacks and code snippets.
- **Test Case Generation:** Automatically generated JUnit 5 test cases for `StudentDAO` and `GradeCalculationStrategy`, covering boundary conditions.
- **Architecture Advice:** Answered questions on organizing MVC package structure and implementing grade distribution statistics.

(3) Specific problems solved by the tool (provide at least Two detailed examples)

Example 1 – Chinese characters displayed as boxes in the grade distribution chart

When implementing `StatisticsChartPanel`, all Chinese text on the chart appeared as “□□□□”. After asking ChatGPT, the solution was to explicitly set a Chinese font (e.g., Microsoft YaHei) for the chart title, legend, and axis labels. The AI provided a complete code snippet that resolved the issue immediately.

Example 2 – Runtime error: `inStream` parameter is null in `DBUtil`

The program crashed with `inStream` parameter is null because the `db.properties` file was not found. ChatGPT analyzed the cause (resource file not in classpath) and offered two solutions: hardcode the database configuration (quick fix) or properly configure the classpath. The hardcode approach was adopted, avoiding complex classpath setup and allowing the program to run immediately.

4. Summary on AI-Assisted Programming:

(1) Describe the benefits brought by AI tools during your development process.

- **Greatly increased coding efficiency:** Routine code (CRUD, Swing layout, JDBC) was generated quickly, allowing focus on business logic.
- **Reduced learning curve:** For unfamiliar libraries (JFreeChart, iText), AI provided ready-to-run examples, reducing documentation time.
- **Automated test generation:** AI produced test cases covering normal and edge inputs, improving code quality.

(2) Analyze the limitations or issues encountered with AI tools and provide suggestions for improvement based on your experience.

Limitations:

- **Limited context understanding:** The AI does not fully grasp the whole project architecture, sometimes generating code that is inconsistent with existing modules or missing dependencies.
- **Occasional errors in generated code:** Type mismatches, missing imports, variable name confusion – manual correction is still needed.
- **Sensitive to local environment:** AI-suggested classpath commands may not work on all systems (e.g., spaces in Windows paths).

Suggestions for Improvement:

- **Ask specific sub-questions:** Provide sufficient context (code snippets, project structure) to reduce errors.
- **Review and refactor:** Treat AI-generated code as a draft; refactor and write unit tests to ensure correctness.
- **Combine with IDE auto-completion:** Use IDE features for imports and refactoring after the AI produces a skeleton.